Questions+ Bonus

Part01

Q1: Why is it better to code against an interface rather than a concrete class?

Answer: Coding against an interface decouples client code from specific implementations. This makes the system loosely coupled, easier to unit test, and flexible to extend without modifying existing concrete code.

Q2: When should you prefer an abstract class over an interface?

Answer: Prefer an abstract class when creating closely related classes (a strong "IS-A" relationship) that need to share common state (fields), protected members, or default concrete implementation code.

Q3: How does implementing IComparable improve flexibility in sorting?

Answer: Implementing IComparable provides a standardized method (CompareTo) that allows built-in framework algorithms like Array.Sort() or List.Sort() to automatically sort custom objects without requiring custom sorting logic every time.

Q4: What is the primary purpose of a copy constructor in C#?

Answer: The primary purpose is to create a new object instance initialized with duplicate values from an existing object instance, preventing side effects caused by shared references.

Q5: How does explicit interface implementation help in resolving naming conflicts?

Answer: It prevents method signature collisions when a class implements multiple interfaces containing methods with identical names, or when a class has its own method with the same signature.

Q6: What is the key difference between encapsulation in structs and classes?

Answer: Memory allocation and value vs reference semantics. Structs are value types stored on the stack and passed by value (copying data), whereas classes are reference types stored on the heap and managed by references.

Q7: How does constructor overloading improve class usability?

Answer: It provides developers flexibility when instantiating objects, enabling creation with full parameters, partial parameters, or default fallbacks without writing extra setup code.

Part02:

Q1: What do we mean by coding against interface rather than class? And what do we mean by code against abstraction not concreteness?

Answer: Both principles emphasize relying on contracts (interfaces or abstract classes) rather than specific class implementations. This reduces tight coupling, enabling software components to be swapped dynamically at runtime without breaking dependent client code.

Q2: What is abstraction as a guideline and how we can implement this through what we have studied?

Answer: As a guideline, abstraction simplifies software design by reducing complexity and hiding internal operational details. We implement it through:

interface: Defining pure behavioral contracts without internal state.

abstract class: Creating base templates with shared logic and enforced abstract method signatures.

Encapsulation: Using private fields with public Properties (get/set) to expose functionality safely

Part03 Bonus:

Q1: Part03 Bonus - Self-Study Report: Advanced OOP Concepts in C#

Answer:

* Abstract Classes vs. Interfaces: Abstract classes support partial implementation, constructors, private/protected state (fields), and single inheritance for closely related types ("IS-A"). Interfaces support contract definitions, multiple inheritance, and stateless designs ("CAN-DO").

* Encapsulation & Access Modifiers: Protects data integrity by restricting direct member access using private or protected fields and exposing them safely through public Properties with validation logic.

* Polymorphism Dynamics: Static Polymorphism (Compile-time) is achieved through Method/Operator Overloading. Dynamic Polymorphism (Runtime) is achieved through Method Overriding using virtual and override keywords.

Q2: Part03 Bonus - What is Operator Overloading?

Answer: Operator Overloading allows C# developers to redefine or customize the behavior of built-in operators (such as +, -, ==, !=) for custom types (classes or structs).

* Rules: Must be declared as public static methods using the operator keyword, and comparison operators must be overloaded in pairs.

Self Question 1: What other allowed Access Modifiers inside an Interface? (C# 8.0+)

Starting from C# 8.0, interface members can have explicit access modifiers:

* public: Default for standard declared interface members.

* private: Allowed only for helper methods containing default code implementation inside the interface.

* protected: Allowed for members meant to be accessed only by derived interfaces or implementing classes.

* internal: Limits access to members within the same assembly.

Self Question 2: How is memory allocated for an Interface reference?

An interface itself cannot be instantiated using new (no object created directly on the Heap).

* Stack: Declaring an interface variable (e.g., IMyType type;) allocates 4 or 8 bytes on the Stack for a reference pointer.

* Heap: The interface reference points to a concrete class object allocated on the Heap that implements the interface (e.g., IMyType type = new MyType();).